



פרויקט מס' BS-CS-26-160

Autonomous Drone Landing on a Moving Target Using Visual Servoing

הנחתת רחפן בצורה אוטונומית על גבי משטח דינמי

פרויקט הנדסי/מחקרי

דוח סיום

הוכן לשם השלמת הדרישות לקבלת

תואר ראשון במדעי המחשב B. Sc.

מאת

טל אפרים ת.ז. 207073636

דניאל רוסמן ת.ז. 211765565

בהנחיית

פרופסור שלמה גרינברג

מר עידן טוביס

הוגש למחלקה למדעי המחשב

המכללה האקדמית להנדסה סמי שמעון

אשדוד

תאריך לועזי

25/08/2026

תאריך עברי

י"ב באלול התשפ"ו



Autonomous Drone Landing on a Moving Target Using Visual Servoing

הנחתת רחפן בצורה אוטונומית על גבי משטח דינמי

פרויקט גמר

דוח מסכם

הוכן לשם השלמת הדרישות לקבלת

תואר ראשון במדעי המחשב B. Sc.

מאת

טל אפרים ת.ז. 207073636

דניאל רוסמן ת.ז. 211765565

הוגש למחלקה למדעי המחשב

המכללה האקדמית להנדסה סמי שמעון

אשדוד

תאריך: 25/08/2026

חתימת סטודנט 1: efraim

תאריך: 25/08/2026

חתימת סטודנט 2: [Signature]

תאריך: _____

חתימת המנחה: _____

תאריך: _____

אישור ועדת הפרויקטים: _____

1. Abstract

Autonomous UAV Landing on a Dynamic Platform Using Computer Vision, Reinforcement Learning, and Sensor Integration

In recent years, Unmanned Aerial Vehicles (UAVs) have become an integral part of many industries, including defense, rescue, logistics, and agriculture.

Despite significant progress in autonomous flight, landing a UAV on a moving target remains a complex engineering challenge due to platform motion, environmental uncertainty, and changing visual conditions.

This project proposes an autonomous landing solution that combines computer vision, sensor integration, and reinforcement learning.

The vision layer is responsible for target detection, re-identification, and tracking, while the reinforcement learning agents learn control decisions through a reward-and-penalty mechanism.

This approach allows the system to address a complex task without manually defining detailed control logic for every possible scenario, instead learning the desired behavior through experimentation, feedback, and interaction with the environment.

The system architecture was originally designed for deployment on physical hardware. However, due to hardware availability limitations and uncertain delivery times following military operation "Shagat Haari," development was shifted to an advanced simulation environment based on Unreal Engine and Cosys-AirSim.

The architecture remains suitable for future hardware integration by replacing simulated observation sources with equivalent data obtained from real sensors and the flight controller.

As part of the project, the computer vision models, tracking mechanisms, two reinforcement learning agents, and a Ground Control Station were developed and integrated into a complete system providing real-time video and telemetry.

The complete system was evaluated in autonomous landing scenarios on a moving target under different target sizes, initial altitudes, and environmental conditions.

The results demonstrated high landing success rates and consistent landing accuracy across the tested operating range. By the end of the project, all major components were integrated into a complete end-to-end system capable of demonstrating fully autonomous landing on a moving target.



תקציר:

הנחתת רחפן בצורה אוטונומית על גבי משטח דינמי באמצעות ראייה ממוחשבת, למידת חיזוק ואינטגרציה עם חיישנים.

בשנים האחרונות, כלי טיס בלתי מאוישים, רחפנים וכטב"מים, הפכו לחלק בלתי נפרד ממגוון תעשיות, החל מביטחון, חילוץ והצלה ועד לוגיסטיקה וחקלאות.

למרות ההתקדמות המשמעותית ביכולות הטיסה האוטונומיות, הנחתת רחפן על מטרה נעה נותרה אתגר הנדסי מורכב, בין היתר עקב תנועת הפלטפורמה, חוסר ודאות סביבתי ותנאי ראייה משתנים.

פרויקט זה מציע פתרון לנחיתה אוטונומית על גבי משטח בתנועה באמצעות שילוב בין ראייה ממוחשבת, חיישנים ולמידת חיזוק. שכבת הראייה הממוחשבת משמשת לזיהוי, זיהוי חוזר ומעקב אחר המטרה, בעוד שסוכני למידת החיזוק לומדים לקבל החלטות מתוך מנגנון של פרסים וקנסות.

גישה זו מאפשרת להתמודד עם משימה מורכבת מבלי להגדיר מראש לוגיקה מפורטת לכל מצב אפשרי, אלא לאפשר למערכת ללמוד את ההתנהגות הרצויה מתוך ניסוי, משוב ואינטראקציה עם הסביבה.

ארכיטקטורת המערכת פותחה במקור עבור חומרה אמיתית, אך עקב מגבלות מלאי וזמני אספקה לא ידועים בעקבות מבצע "שאגת הארי", שונתה האסטרטגייה לפיתוח בסביבת סימולציה מתקדמת המבוססת על Unreal Engine ו־ Cosys-AirSim. אך ארכיטקטורת המערכת עדיין מתאימה לאינטגרציה עתידית עם חומרה באמצעות החלפות מקור נתוני ה- observations שמוזנים למודל, מנתוני סימולציה לנתונים שמתקבלים מחיישנים אמיתיים ובקר הטיסה.

במסגרת הפרויקט פותחו מודלי הראייה, מנגנוני המעקב, שני סוכני למידת החיזוק וממשק תחנת שליטה קרקעית המציג וידאו ונתוני טלמטריה בזמן אמת.

המערכת השלמה נבחנה בתרחישי נחיתה על משטח בתנועה, תחת גדלי מטרה, גבהים ותנאי סביבה שונים. תוצאות הניסויים הראו שיעורי הצלחה גבוהים ודיוק נחיתה עקבי לאורך טווח התנאים שנבדק.

בסיום הפרויקט שולבו כלל רכיבי המערכת – לוגיקת הטיסה, מודלי הזיהוי והמעקב, סוכני הלמידה ותחנת השליטה – לכדי מערכת אחת הפועלת מקצה לקצה ומדגימה נחיתה אוטונומית מלאה על מטרה בתנועה.

2. Table of Contents

1.	Abstract	
2.	Table of contents	
3.	Introduction	
4.	Project objectives	
5.	System description	
5.1	System architecture overview	
5.2	Hardware Components	
5.3	Visual representation of the drone	
5.4	Software Components	
5.5	GUI Prototype Visualization	
6.	Methodology and Algorithms	
7.	Final system status and performances	
8.	Challenges and Roadblocks	
9.	Summary and Conclusions	
10.	Testing and model limitations research	
11.	Link to project files on GITHUB	

3. Introduction

In recent years, Unmanned Aerial Vehicles (UAVs) have become widely used in fields such as defense, agriculture, logistics, monitoring, and search and rescue. Advances in autonomous flight have enabled UAVs to perform increasingly complex tasks with reduced human intervention.

Despite this progress, autonomously landing a UAV on a moving platform remains a challenging engineering problem. The system must continuously detect and track the target, estimate its relative motion, align the UAV above the landing area, control the descent, and verify that a safe landing has been achieved. These tasks become more difficult when the target is moving, when visual information is partially lost, or when environmental conditions change during the flight.

This project addresses the problem by combining computer vision, reinforcement learning, motion prediction, and sensor-based feedback into a complete autonomous landing system.

The system is divided between two cooperative reinforcement learning agents. The first agent is responsible for target tracking, horizontal alignment, and yaw correction, while also determining when the UAV has reached a stable landing-ready position. Once these conditions are satisfied, the second agent takes responsibility for the vertical descent, altitude control, descent-rate adjustment, and the final landing phase.

The perception layer continuously processes visual information in order to detect the selected target, maintain its identity, and estimate its position during flight. When direct visual confirmation becomes temporarily unreliable, motion prediction is used to support the tracking process. During the final descent, visual information is combined with range sensors and contact feedback in order to maintain stability and verify the landing.

The complete system was developed and evaluated in Unreal Engine and Cosys-AirSim under different target sizes, initial altitudes, motion conditions, wind conditions, and visual environments. In addition, a custom React-based Ground Control Station was developed to provide real-time video, telemetry, system status, and mission controls.

The final system was evaluated through autonomous landing experiments and demonstrated high tracking reliability, consistent landing accuracy, and a high overall landing success rate across the tested operating range.

4. Project Objectives

The primary objective of this project is to develop an autonomous UAV landing system capable of detecting, tracking, approaching, and landing on a moving target using a combination of reinforcement learning, computer vision, motion prediction, and sensor feedback.

The main project objectives are:

- Develop and train reinforcement learning agents in a simulation environment under different operational conditions.
- Develop a vision-based target detection and tracking system capable of maintaining target identity throughout the flight.
- Enable the UAV to autonomously align itself with a selected moving target and transition into a controlled landing phase.
- Use sensor feedback during the final descent in order to support stable and reliable landing behavior.
- Integrate the flight-control logic, perception components, learning models, and Ground Control Station into a complete end-to-end system.
- Evaluate the complete system through repeated landing experiments under different target sizes, initial conditions, and environmental conditions.

In addition, several system-level objectives were defined:

- Develop a computer interface that displays the UAV's Point of View (POV), telemetry, and system status.
- Allow the user to select a potential static or dynamic landing target through the Ground Control Station.
- Provide manual flight-control capabilities through the user interface.
- Provide an autonomous target-tracking and landing mode.
- Maintain communication between the simulation environment, the control system, and the Ground Control Station.

Success Metrics

The performance of the system is evaluated using several complementary metrics:

- **Landing success:** Maximize the percentage of completed and validated autonomous landings.
- **Landing accuracy:** Minimize the horizontal deviation between the UAV landing position and the center of the selected target.
- **Target tracking reliability:** Maximize the amount of time in which the system maintains a reliable visual lock on the selected target.
- **Landing-stage perception:** Maintain reliable target awareness during the transition from tracking to final descent.
- **Landing preparation time:** Minimize the time required for the UAV to reach a stable landing-ready state.
- **Total landing time:** Minimize the total time from target selection until successful contact and landing validation.
- **Robustness:** Maintain stable system behavior under different target sizes, initial conditions, motion patterns, and environmental disturbances.
- **System integration:** Successfully operate the perception, control, reinforcement learning, sensing, and Ground Control Station components as one complete end-to-end system.
- **Target-selection functionality:** Allow the user to select a landing target through the interface without requiring prior knowledge of the target's position.
- **Control stability:** Maintain stable UAV motion during tracking, alignment, descent, and final contact with the landing surface.

Project Vision:

The vision of this project is to develop a generic and modular framework for autonomous UAV landing on moving targets using reinforcement learning, computer vision, motion prediction, and sensor-based feedback.

The system is designed to reduce the need for manually defined control logic by allowing the UAV to learn how to track, align with, and land on a selected target through interaction with the environment and continuous feedback.

A high-fidelity simulation environment is used as the main development and evaluation platform, allowing the complete system to be trained, tested, and analyzed under repeatable conditions before considering deployment on physical hardware.

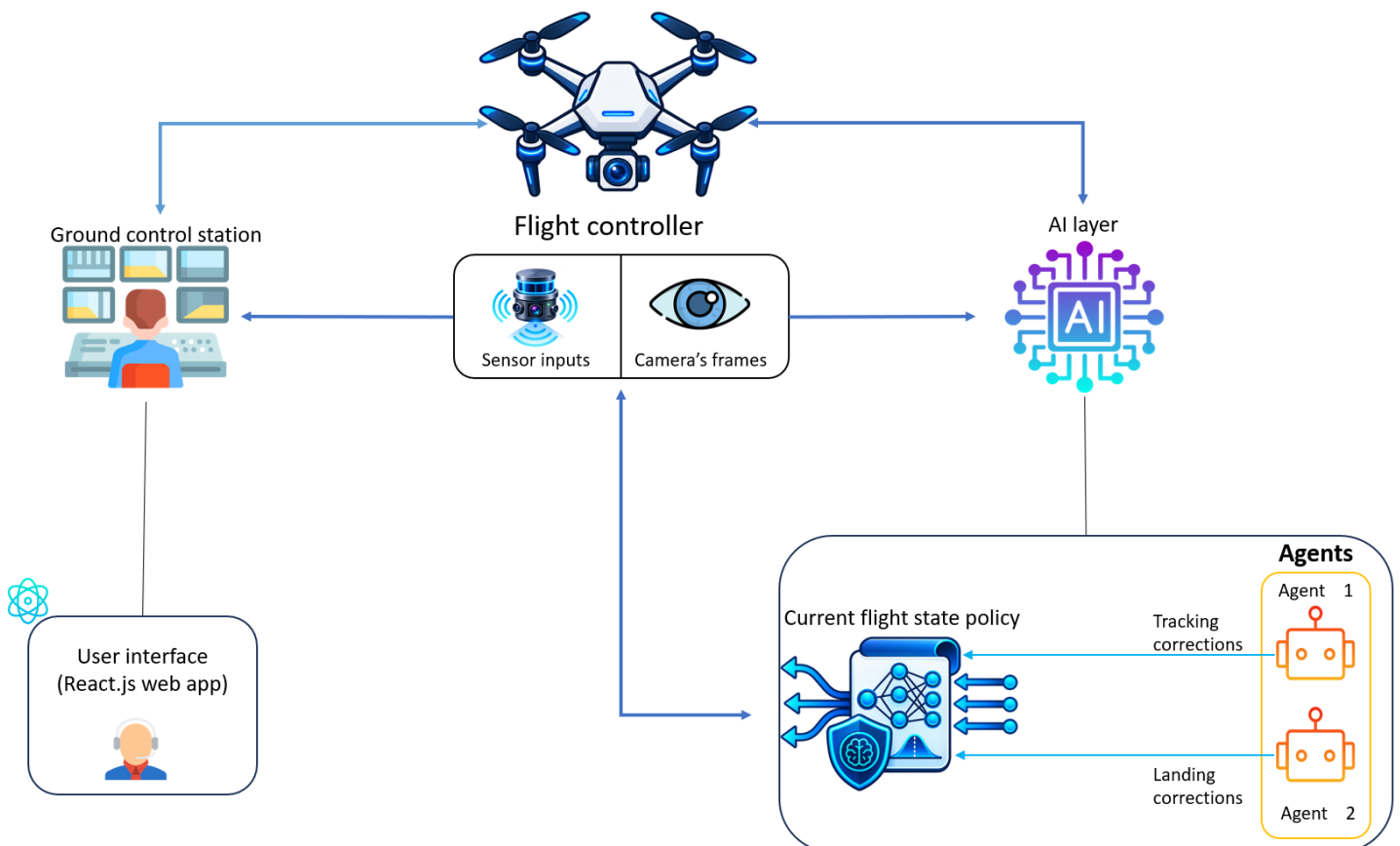
The proposed architecture is intended to remain flexible and reusable, so that the same general approach can be adapted in the future to different UAV platforms, sensors, and operational environments.

The long-term vision is to enable UAVs to autonomously perceive a selected target, estimate its motion, maintain reliable tracking, align themselves for landing, and complete a controlled landing with minimal human intervention.

Rather than focusing only on a single landing scenario, the project aims to demonstrate a broader robotics approach in which complex autonomous behavior can be achieved through the combination of learning, perception, sensing, and feedback.

5. System Description

5.1 System architecture in high level:





5.2 Planned Hardware Components

While the current phase of the project is entirely simulation based, the simulation framework is specifically designed to model a designated physical architecture. The table below details the target hardware components planned for the future real-world implementation of the UAV. The algorithmic development, computational constraints, and telemetry workflows within the simulation are tailored to align with the specifications of these specific edge devices (NVIDIA Jetson Orin NX and Pixhawk).

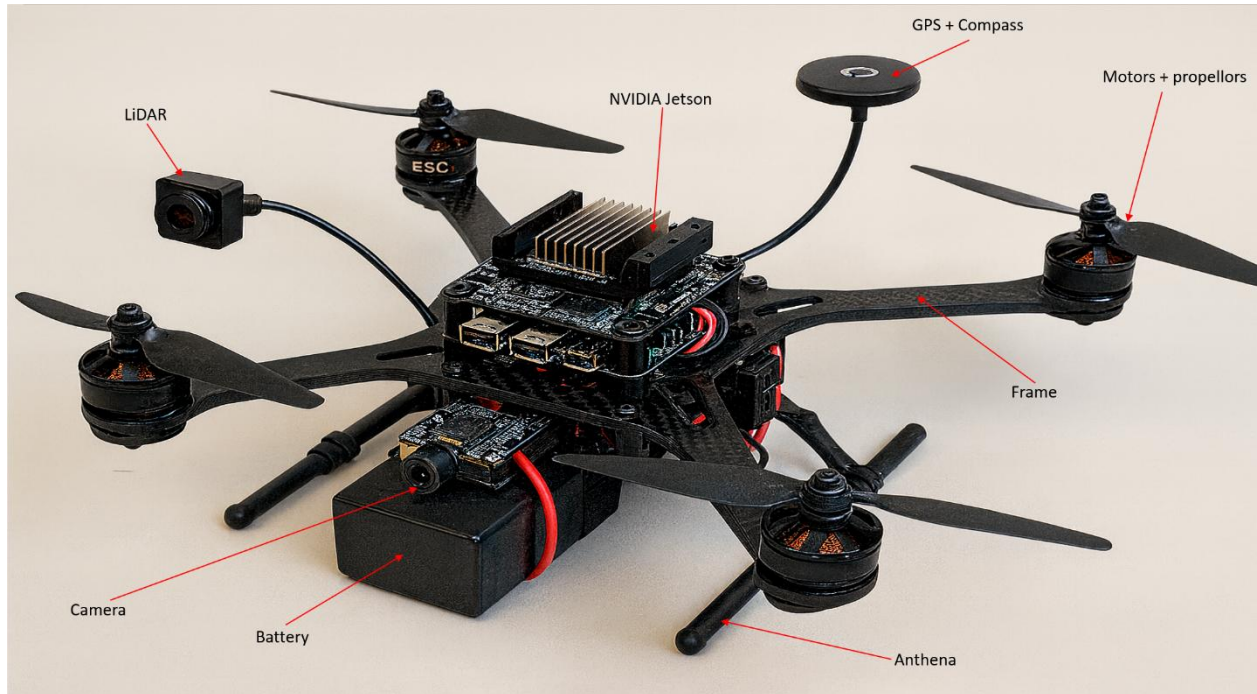
Hardware Component	Function / Description	Specifications / Notes
Digital camera (e-CAM86_CUONX)	Capturing high quality images for image processing	low latency
Low freq modem (transceiver)	streaming drone's telemetry and control it in real time	915MHz/433MHz
915MHz / 433MHz antenna (x4)	streaming the telemetry and RC commands	Tx/Rx diversity
High freq cellular modem (transceiver)	Main video stream	5G
RTC (Real-Time Clock) for Jetson	Keeping time synchronized after power shutdown	
5G antenna (x4)	streaming real-time video to the operator in MIMO 4x4 protocol	5G
Frame	drone chassis	Quadcopter frame
Protecting boxes for sensitive modules	Need to protect modems and the Jetson	
Rubber / Foam Padding	Isolation and vibration dampening.	1 for each HW component and 1 for each leg.
Status LED	Indication lights for various drone status updates.	RED, Yellow, Green and Blue
Kill Switch	Emergency neutralization switch. Stops the drone in case of a safety issue. Will be connected to the Ground Station transmitter.	Bypasses all systems and neutralizes the drone according to a pre-defined safety protocol.
Propellers	Blades connecting to the motors to generate lift.	TBD Capacity to be determined based on final simulation energy metrics.
Motors	Providing power to the propellers	TBD Capacity to be determined based on final simulation energy metrics.



Battery	Power supply for the drone.	TBD Capacity to be determined based on final simulation energy metrics.
Step-down Converter	Voltage reducer for low-voltage components	Reduce to 5v and less
NVIDIA Jetson Orin NX 16GB	Onboard computer for running Computer Vision algorithms.	
Connect Tech Sprocket Carrier Board	Carrier board for Jetson	
Flight Controller (Pixhawk)	Drone stabilization, navigation, sensor processing, flight modes, communication, and motor control.	PX4 based
GPS + Compass	Provides general spatial positioning and orientation of the drone.	
LiDAR	High-precision distance measurement relative to sensor orientation.	(We might need one for each direction).
Large Electrolytic Capacitors	Prevent noises and voltage spikes	TBD Capacity to be determined based on final simulation energy metrics.
Power Distribution Board (PDB)	Distributes voltage to motors and provides current feedback to the PIXHAWK	Might be included with the Pixhawk flight controller; needs verification.
Cables Kit	Cable kit for connecting all hardware components.	by default and which need to be purchased separately.

**This section details the target hardware architecture initially designed to support the real time computer vision algorithms. Due to budgetary constraints and procurement delays regarding the specific UAV platform, our final deliverable has transitioned to a fully functional model operating within a Desktop App simulation environment. Consequently, the physical hardware assembly phase has been deferred to future stages of development, allowing us to first rigorously validate and optimize the software architecture within the simulation.

5.3 Visualization of Target Drone Components



Note 1: Scope of Representation The accompanying visualization highlights the primary hardware interfaces and key subsystems. While this rendering does not explicitly display every item listed in the Bill of Materials (BOM) provided in the preceding section, all specified components including auxiliary hardware and connectors will be integrated into the final physical assembly.

Note 2: Conceptual Design Disclaimer This rendering is intended for illustrative purposes to demonstrate the approximate spatial topology of the system. It is important to note that the final mechanical design and specific component placement are subject to modification during the detailed engineering and planning phases to optimize weight distribution, cabling efficiency, and structural integrity.

Note 3: Specific Component Rendering For the purpose of this visualization, the **Jetson Orin Nano** computing module has been explicitly rendered to demonstrate its integration within the frame. This inclusion serves strictly as a visual reference for the onboard processing unit's form factor and location.

5.4 Software Components

As outlined in the initial project characterization and the definition of primary and secondary objectives, the UAV system operates under two distinct functional modes:

- **Manual Mode:** Direct manual control of the simulated UAV via a designated input device (keyboard, virtual joystick, or mapped external controller) interfacing with the simulation engine.
- **Fully Autonomous Mode:** The simulated UAV assumes full control authority, autonomously managing flight parameters including position, altitude, and navigation based on the integrated algorithmic logic.

Override Protocol: The Ground Control Station (GCS) user interface is developed as a Desktop application utilizing React electron library, with the visual styling architecture implemented via CSS libraries. This approach provides a responsive and dynamic control environment.

User Interface (UI) Implementation:

The user interface will be developed using **JavaScript**, with the visual styling architecture implemented via **CSS** libraries.

The Interface Dashboard includes the following modules:

- **Main Dashboard Frame:** A primary container hosting all interface objects, elements, and menus.
- **UAV Attitude (Orientation):** Real-time visualization of the simulated drone's pitch, roll, and yaw.
- **System Diagnostics:** General status indicators for the active algorithms and simulation backend.
- **Simulated Signal Strength (RSSI):** Virtual indicators representing connection quality between the GCS and the simulated drone.
- **Flight Mode Status:** Current operational state indicator (Manual / Auto / Hover).

Primary HUD (Heads-Up Display) Widget: A central widget displaying the real time simulated POV (Point of View) video feed rendered by the Unreal Engine environment. Overlaid on this feed, the following telemetry data generated dynamically by the simulation will be presented:

- Simulated Altitude AGL (Above Ground Level) bar.
- Virtual signal reception strength indicator.
- Simulated battery status percentage.

- Virtual airspeed.
- Simulated GPS Coordinates (Latitude/Longitude).

Target Acquisition Interface: A dynamic bounding box for navigation targets within the simulation, which displays a red "LOCKED" indicator when the computer vision system actively engages in visual object tracking.

Command Control Panel: A dedicated button menu for high-level UAV commands, supporting functions such as:

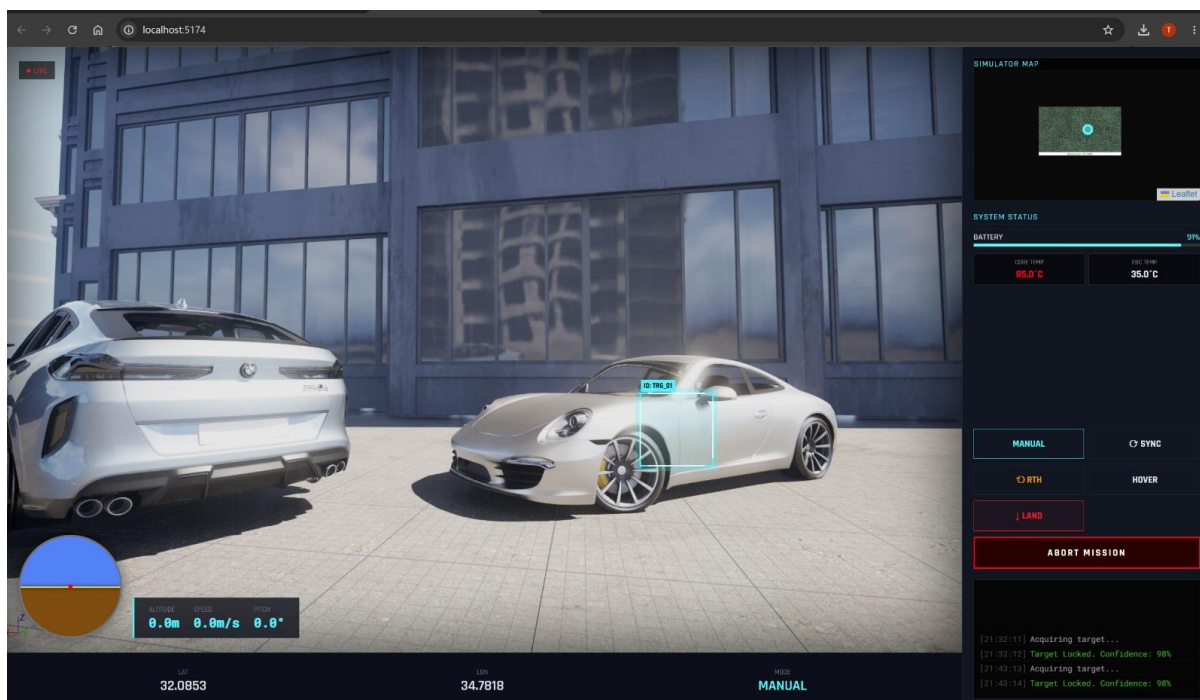
- "Follow Target"
- "Hover"
- "Return to Home" (RTH) – directing the UAV back to the Ground Control Station coordinates. (or user's custom defined location)

System Logs & Tracing Console: A text-based window displaying real time system status messages categorized by severity: **Info, Warning, Error, and Fatal.**

Secondary Navigation Map: A compact widget displaying a 2D map interface. This visualizes the UAV's real time GPS position relative to its launch point (Home Point).

5.5 GUI Prototype Visualization

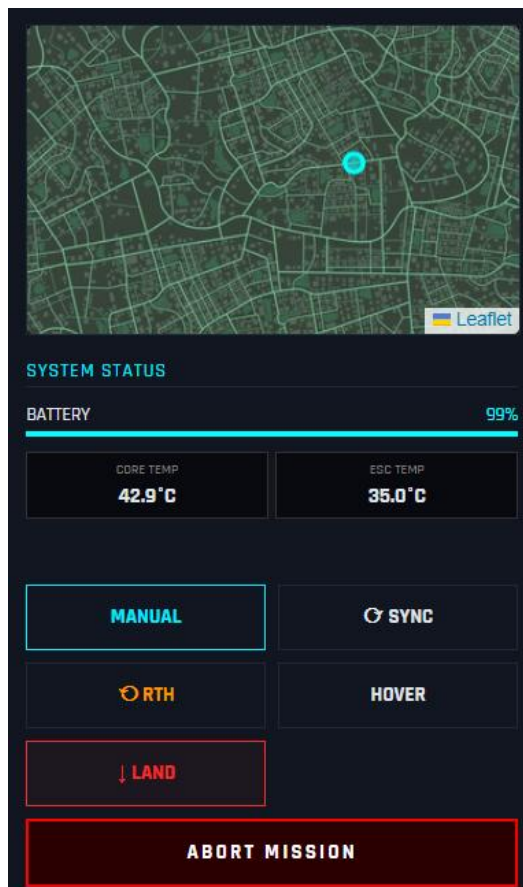
main frame



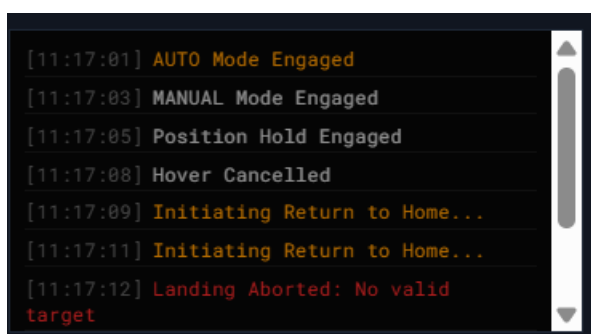
Camera display and speedometers



Mini map and features



Logs and Abort button



6. Methodology and Algorithms

The main idea is taking a raw RL network and using computer vision technics and well-planned reward system, to train the drone for two missions:

- follow the target
- land on the target

There are two separate agents, that each is experts for Its specific task.

The algorithmic core is combined from two parts:

1. Collecting target deep embedded features for id signature
2. Tracking the target with custom tracker (YOLO for signature, ResNet18 for reid), which comparing the embedded features for tracking, and Kalman filter for prediction in case of target lost while flying.

The core of the model has 37 channels (learning weights) as follows:

The updated observation vector is composed of **37 normalized channels**.

Compared to the previous larger architecture, the current design is more compact and focuses only on information that is immediately useful for visual tracking, stable flight, obstacle awareness, and future landing logic.

The observation channels are divided into 5 logical groups:

1. Target / vision core
2. Image-space motion and visual range
3. Drone motion / attitude
4. Mission and tracking-state logic
5. Obstacle and safety awareness

Unlike the older 48-channel proposal, the current 37-channel version does not include a dedicated hardware-only reserve block or explicit control-history block. This keeps the observation space smaller, easier to train, and compatible with the current PPO model architecture.

A. Target / vision core block — 7 channels

These channels describe whether the target exists in the image and its basic bounding-box geometry.

Idx	Channel	Why
-----	---------	-----

- | | | |
|---|------------|--|
| 0 | has_target | indicates whether a valid target is currently detected/tracked |
| 1 | bbox_cx | normalized target center X position |
| 2 | bbox_cy | normalized target center Y position |
| 3 | bbox_w | normalized target bounding-box width |
| 4 | bbox_h | normalized target bounding-box height |
| 5 | bbox_area | visual size of the target in the image |
| 6 | bbox_conf | tracker confidence / bbox reliability |

Why this matters

This block gives the policy the basic visual state of the target:

- whether the target is visible
- where the target is located in the image
- how large the target appears
- how reliable the current visual detection is

This is the core information required for visual servoing and target-following behavior

B. Image-space motion and visual range block — 9 channels

These channels describe target centering error, target motion in the image, and visual distance change.

Idx Channel	Why
7 err_x	horizontal image error from the center
8 err_y	vertical image error from the center
9 img_vx	target horizontal image velocity
10 img_vy	target vertical image velocity
11 img_ax	target horizontal image acceleration
12 img_ay	target vertical image acceleration
13 area_delta_norm	change in target visual area
14 distance_proxy_norm	visual distance proxy based on bbox size
15 distance_proxy_delta	change in visual distance proxy

Why this matters

This block allows the policy to understand not only where the target is, but also how it is moving in the camera frame.

It helps the agent learn:

- yaw correction
- vertical correction
- target motion prediction
- whether the drone is visually approaching or drifting away
- smoother visual following

The distance_proxy_norm is useful for basic visual-follow behavior, but it should not be treated as a true physical distance measurement. For real range-based approach or landing logic, physical distance should come from simulator ground truth with simulated sensor noise, or from real hardware sensors in deployment.

C. Drone kinematics / attitude block — 8 channels

These channels describe the physical motion and attitude of the drone.

Idx Channel	Why
16 altitude_norm	normalized altitude above ground
17 drone_vx_norm	normalized drone X velocity
18 drone_vy_norm	normalized drone Y velocity
19 drone_vz_norm	normalized drone vertical velocity
20 drone_roll_norm	normalized roll angle
21 drone_pitch_norm	normalized pitch angle
22 drone_yaw_rate_norm	normalized yaw-rate
23 drone_speed_xy_norm	normalized horizontal speed

Why this matters

This block is critical for stable physical control.

It helps the policy understand:

- current drone speed
- vertical movement
- altitude state
- yaw dynamics
- roll and pitch attitude
- whether the drone is moving smoothly or aggressively

This is especially important for sim-to-real transfer, because the policy must not rely only on image information. It also needs awareness of the drone's own motion state.

D. Mission and tracking-state logic block — 4 channels

These channels summarize high-level tracking quality and mission readiness.

Idx Channel	Why
24 centered_score	how well the target is centered
25 target_stability_score	how stable the target is over time
26 landing_allowed	indicates whether target state is suitable for landing logic
27 lost_target_time_norm	normalized time since target was lost

Why this matters

This block turns raw visual information into more meaningful mission signals.

It helps distinguish between:

- target visible but unstable
- target centered and reliable
- target lost temporarily
- target lost for too long
- target state suitable for future landing behavior

This is important because the drone should not only react to raw pixels. It should also understand whether the tracking state is reliable enough for more advanced behavior such as approach, descent, or landing.



E. Obstacle and safety awareness block — 9 channels

This block gives the drone local free-space awareness using normalized obstacle-distance signals.

Idx Channel	Why
28 front_dist_norm	frontal obstacle risk
29 front_left_dist_norm	diagonal-left obstacle risk
30 front_right_dist_norm	diagonal-right obstacle risk
31 left_dist_norm	left-side clearance
32 right_dist_norm	right-side clearance
33 back_dist_norm	rear obstacle clearance
34 down_dist_norm	ground / landing clearance
35 min_obstacle_dist_norm	closest obstacle summary
36 collision_risk_score	normalized collision-risk estimate

Why this matters

This block gives the policy safety awareness around the drone.

It allows the network to reason about:

- frontal collision risk
- side clearance
- ground distance
- closest obstacle pressure
- general collision danger

This is especially important for bridge-underpass training, low-altitude flight, and future landing behavior.

Without this block, the policy could learn to keep the target centered while ignoring nearby obstacles, which would be unsafe and unrealistic.

Summary of the 37-channel architecture

Group	Channels	Count	Purpose
Target / vision core	0–6	7	target presence and bbox geometry
Image-space motion and visual range	7–15	9	centering error, image motion, visual approach
Drone kinematics / attitude	16–23	8	physical drone state
Mission and tracking-state logic	24–27	4	tracking quality, stability, landing readiness
Obstacle and safety awareness	28–36	9	obstacle distances and collision risk
Total	0–36	37	compact observation vector

Architectural motivation

The updated 37-channel observation vector is designed to be compact, practical, and trainable.

It focuses on the minimum information required for stable autonomous visual following:

- target position
- target motion
- drone motion
- tracking stability
- obstacle awareness

The main advantage of this design is that it separates visual tracking from physical control.

The vision channels tell the agent:

- where the target is
- whether the target is stable
- whether the target is centered

The drone-state channels tell the agent:

- how the drone is moving
- how high it is
- whether its attitude is stable



The obstacle channels tell the agent:

- whether the current motion is safe
- whether the drone is near the ground or an obstacle

This makes the architecture suitable for step-by-step curriculum learning:

1. stable target tracking
2. smooth following
3. obstacle-aware tracking
4. bridge-underpass behavior
5. future approach and landing logic

For future physical approach and landing, real distance, velocity, and range-rate should be introduced as physical sensor signals, using simulator ground truth with configurable noise. This keeps the architecture realistic for future hardware deployment, while avoiding the mistake of treating bounding-box size as a true physical distance sensor.

Tracker Evolution: From Histograms to Deep Embeddings

Old Architecture vs. New Architecture: Enhancing Object Tracking for RL

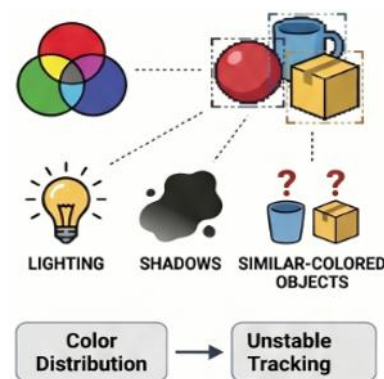
Old tracker architecture: Histogram-Based

Relies on color distribution; highly sensitive to lighting, shadows, and similar-colored objects.

Color Distribution → Unstable Tracking.

Performance is easily disrupted by:

- Lighting
- Shadows
- Similar-colored objects



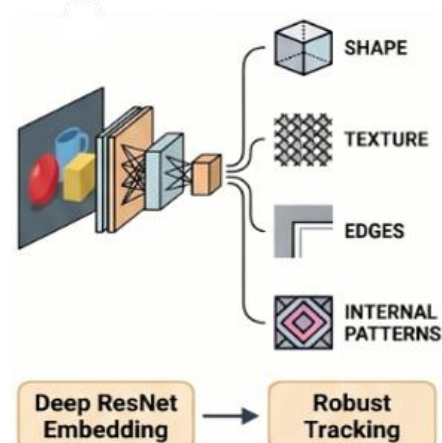
New tracker architecture: YOLO + ResNet

Extracts deep embeddings focusing on shape, texture, edges, and internal visual patterns.

Deep ResNet Embedding → Robust Tracking.

Analyzes multiple complex features, including:

- Shape
- Texture
- Edges
- Internal Patterns



Performance, stability metrics and comparison:

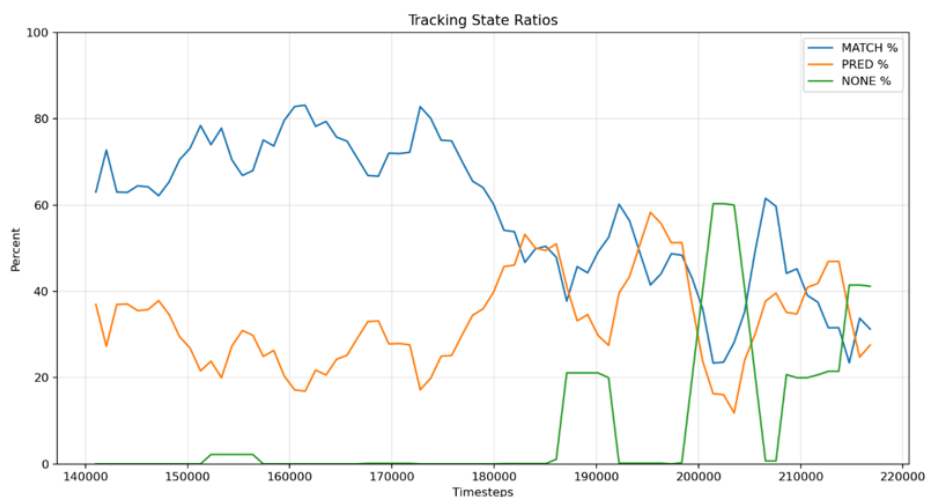
Old Architecture:

- RL Training Stability: Less stable data for RL training.
- Visual Robustness: High sensitivity to environmental changes.

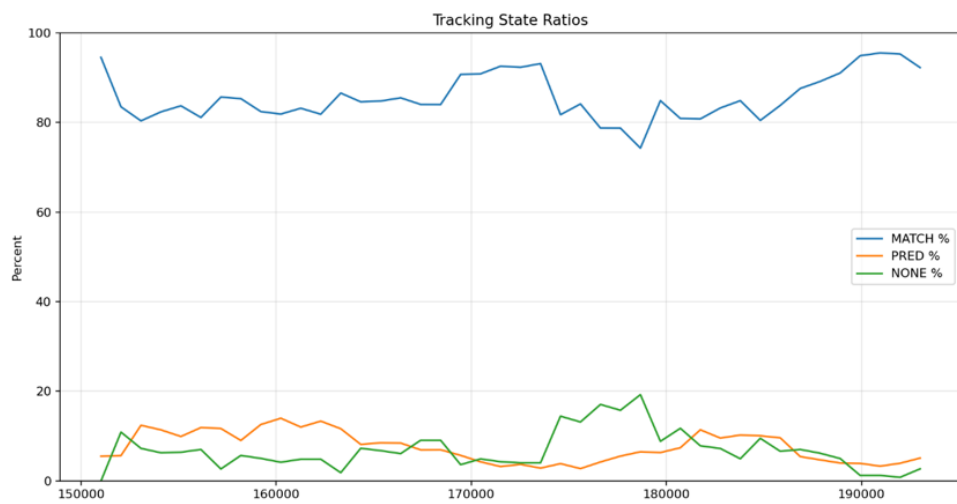
New Architecture:

- RL Training Stability: Significantly more stable data for RL training.
- Visual Robustness: Lower sensitivity to environmental changes (blur, camera angles).

old tracker MATCH-PRED-NONE ratio:



new tracker MATCH-PRED-NONE ratio:



Technical comparison

ID Matching: The old histogram-based architecture relies on color distribution to match objects, whereas the new YOLO + ResNet architecture uses deep ResNet embedding.

Sensitivity: The old architecture has high sensitivity to environmental factors like lighting and shadows. In contrast, the new architecture has low sensitivity because it focuses on structural features like shape and texture.

RL Suitability: The old architecture provides a less stable foundation for Reinforcement Learning (RL) training, while the new architecture is much more stable.

TECHNICAL COMPARISON		
Feature	Old: Histogram-Based	New: YOLO + ResNet
 ID Matching	Color distribution	Deep ResNet embedding
 Sensitivity	High (Lighting/Shadows)	Low (Focus on Shape/Texture)
 RL Suitability	Less stable	Much more stable

PERFORMANCE & STABILITY METRICS			
			
RL Training Stability Less stable data for RL training.	Visual Robustness High sensitivity to environmental changes.	RL Training Stability Significantly more stable data for RL training.	Visual Robustness Lower sensitivity to environmental changes (blur, camera angles).

7. Final system status and performances

Current system state:

The project has successfully reached a complete end to end implementation of an autonomous landing system for a UAV on a moving target.

The final system is based on two cooperative RL agents:

- Agent 1 is responsible for target tracking, horizontal alignment, yaw correction and degerming when the UAV is ready to begin the landing phase.
- Agent 2 is responsible for vertical descent, altitude control, descent-rate adjustments and completing the final landing phase.

Both agents operate together with the computer vision layer and the onboard sensing components in order to maintain target awareness and safely complete the landing process.

Model status and optimizations:

Optimized parameters:

The final models were trained and tuned around main kinematic and control variables required for the task, including horizontal motion, altitude, yaw alignment, descent behavior, velocity control, reward shaping, and the thresholds used to determine when the UAV is ready to transition into landing phase.

These parameters were gradually adjusted throughout the deployment process in order to achieve stable tracking, controlled descent and consistent landing behavior.

Final performance metrics:

The final system was evaluated using 700 autonomous landing trials across seven different landing surface sizes, with 100 trials performed for each target size.

The system was tested in different physical conditions such as varying wind speeds and wind directions, different lighting and weather conditions (sunny, rainy, falling leaves, and light fog)

The complete experimental setup and results are presented in [Testing Section](#)

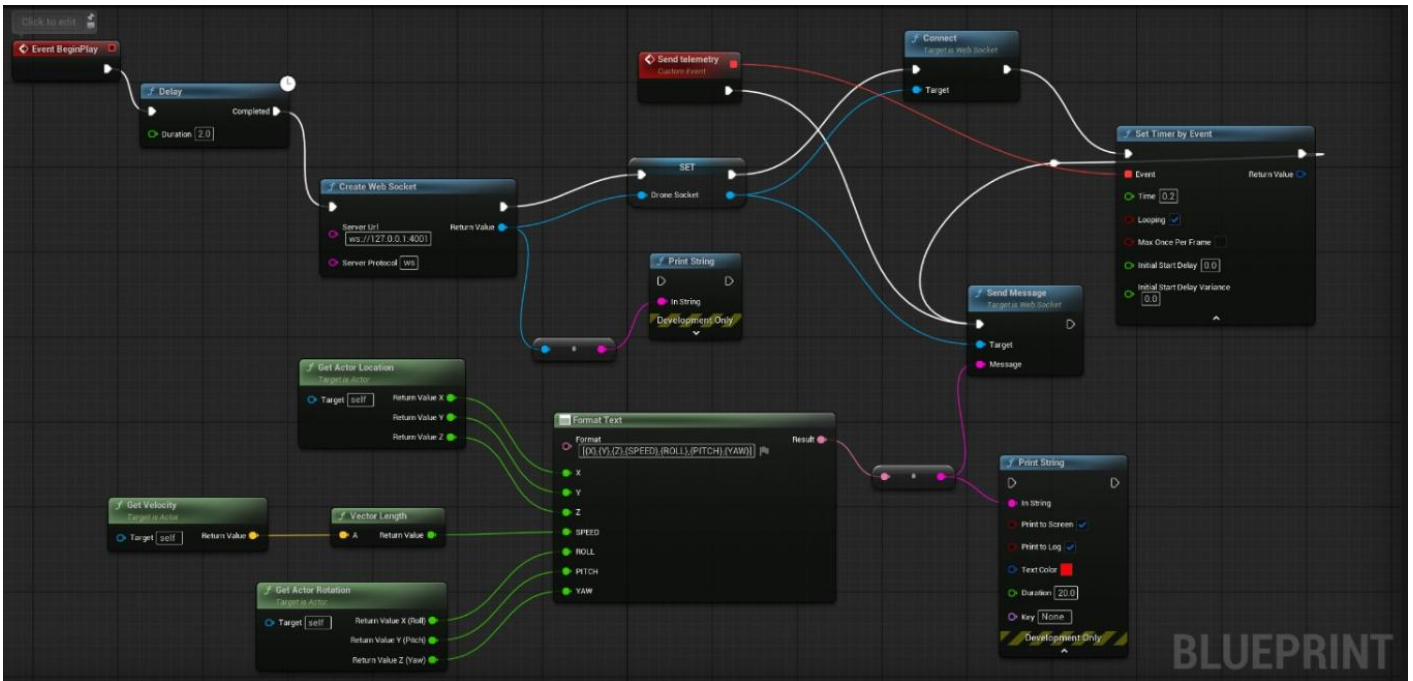
8. Challenges and Roadblocks

While the original project scope anticipated training and integration on physical hardware, our strategic pivot to a comprehensive simulation based deliverable has redefined our integration pipeline. Consequently, constraints related to specific physical UAV platforms and proprietary SDKs no longer impede our core development.

However, establishing the foundational communication between the Unreal Engine simulation environment and the React based Graphical User Interface (GUI) presented significant technical hurdles. We evaluated multiple data transmission methodologies to bridge the simulator and the frontend, including WebSockets, Pixel Streaming, and HTTP requests via dedicated plugins or external APIs. To facilitate this data extraction, we had to engineer a custom Unreal Engine *Blueprint* a node based visual scripting interface used to define object logic and behaviors within the engine specifically for the drone entity.

During initial testing, this integration suffered from noticeable communication latency and data format mismatches between the simulator's telemetry output and the UI's state management. Following extensive debugging and architectural optimization to ensure an efficient communication channel, we ultimately adopted the **AirSim API** as our primary integration layer. This solution successfully established a robust, bi-directional data pipeline, enabling seamless streaming of real time telemetry from the simulator to the React UI, while reliably transmitting control commands from the UI back to the simulated drone.

Ultimately, we successfully resolved these integration challenges, achieving full end to end virtual integration for the autonomous agent. The data pipeline is now fully operational, enabling the dynamic and continuous routing of simulated live telemetry extracting state data such as velocity, altitude, and angular orientation directly into the neural network's observation space to ensure complete spatial awareness. Furthermore, we successfully implemented and validated robust bi directional command execution. The system now reliably processes and executes critical drone flight directives directly from the React UI to the simulation, including **Manual** and **Automatic** flight modes, **Emergency Abort**, **Land in Place**, **Hover**, and **Return to Home (RTH)**.



Initially, the provided visual scripting diagram outlined the execution flow of a real time telemetry transmission system within the 3D simulation environment. Upon system startup, a two-second initialization delay preceded the instantiation of a WebSocket client, configured to establish a connection with a local server. The system utilized a looping timer to trigger a custom telemetry event at 5 Hz intervals, systematically querying the drone's spatial and kinematic states—specifically extracting its three-dimensional spatial coordinates, absolute scalar speed, and rotational orientation. These localized metrics were manually serialized into a structured format and transmitted continuously via WebSockets. However, due to subsequent architectural considerations, the decision was made to deprecate this custom blueprint methodology in favor of utilizing the AirSim API.

This transition was necessitated by the requirement to more seamlessly integrate algorithmic autonomous systems directly with the simulation environment. The AirSim API natively manages complex telemetry data flows and visual configurations, offering a standardized and highly optimized programmatic interface. By abstracting the manual serialization process seen in the blueprints, the API provides superior synchronization and state extraction capabilities, which are critical when continuously streaming real-time kinematic data to external reinforcement learning and computer vision pipelines.

9. Summary and Conclusions

Project Overview and Objective

This final report presents the complete end-to-end project, including its purpose, the engineering problem being addressed, the proposed system architecture, the experiments conducted, the results obtained, and the main conclusions.

Project Purpose:

The purpose of the project is to develop an autonomous UAV landing system capable of detecting, tracking, approaching, and landing on a moving target using reinforcement learning, computer vision, motion prediction, and sensor feedback.

Engineering Problem:

The project addresses a broader robotics challenge: how can an autonomous system learn to perform a complex control task without manually defining detailed control logic for every possible situation and without knowing the target's exact position in advance?

Proposed Architecture

The system is based on two cooperative reinforcement learning agents.

The first agent is responsible for target tracking, horizontal alignment, and yaw correction. Its objective is to position the UAV in a stable landing-ready state relative to the selected target.

The second agent is responsible for the vertical descent and landing phase. It controls altitude and descent rate using downward visual information and range-based sensor feedback while cooperating with the first agent during the final approach.

In-Depth Details:

- Camera frames are processed by the computer vision layer, which detects the selected target, extracts visual features, maintains target identity, and tracks its position over time. When direct visual tracking becomes temporarily unreliable, motion prediction is used to maintain an estimate of the target position.
- The information produced by the perception layer is converted into observations and provided to the control system. Agent 1 uses this information to generate horizontal and yaw corrections required to track and align with the moving target.

- Once the UAV reaches a sufficiently stable and centered landing-ready state, the system transitions into the descent phase. At this stage, Agent 2 takes responsibility for vertical motion, while Agent 1 continues to support horizontal alignment and tracking as required.
- During the descent phase, the system uses the downward-facing camera together with range-sensor information to maintain alignment, regulate descent, and verify successful contact with the landing surface.

The two agents therefore operate cooperatively rather than as completely independent controllers. Agent 1 brings and maintains the UAV in the required horizontal landing state, while Agent 2 controls the vertical descent and final landing process.

Dividing the task between two agents allows each policy to focus on a smaller and more clearly defined control problem instead of requiring a single policy to simultaneously learn tracking, alignment, descent, and landing behavior.

The project was developed and evaluated in simulation. However, the observation architecture was designed so that simulated sensor and vehicle-state inputs can later be replaced by equivalent measurements from physical sensors and a real flight controller. A physical implementation would still require sensor calibration, communication integration, and additional validation or fine-tuning under real-world conditions.

Testing section:

Experimental Setup and Performance Results

Target Configurations: Seven different target surface sizes were evaluated.

Testing Iterations: Each target size was tested across 100 iterations.

Altitude Range: Experiments were conducted at varying heights between 2 and 5 meters.

Dynamic Target Movement: The target moved in a straight line between two towers at an average speed of 30 km/h under varying crosswind conditions.

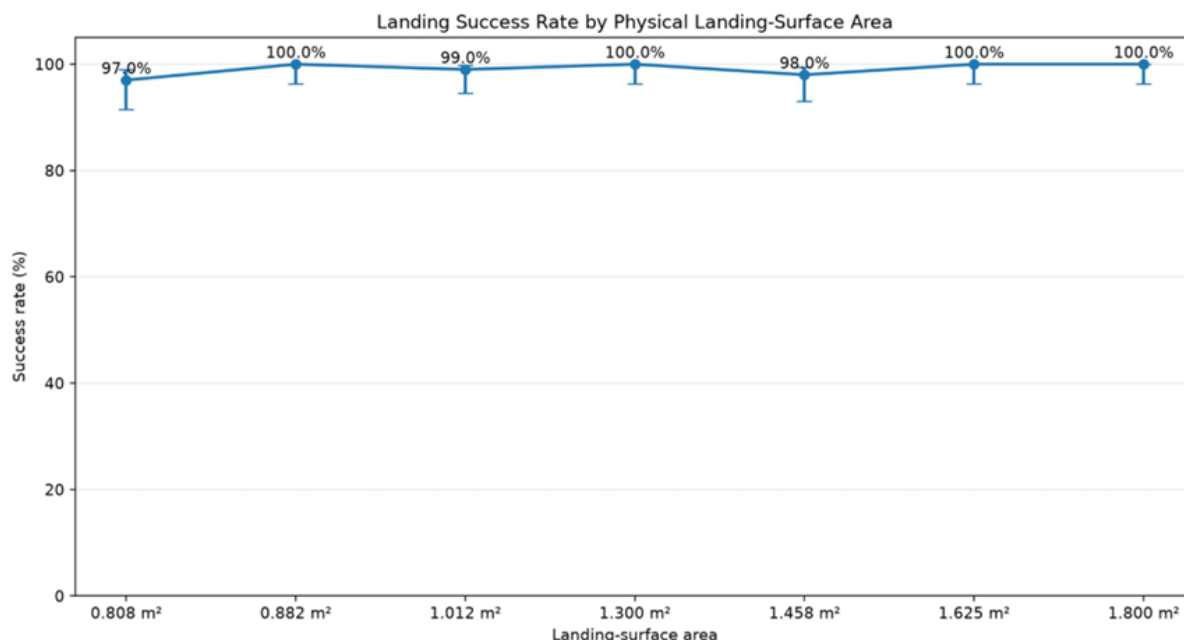
Success Rates by Surface Area:

Surface Area (m ²)	Success Rate
1.800	100%
1.625	100%
1.458	98%
1.300	100%
1.012	99%
0.882	100%
0.808	97%

Key Findings and Limitations

- The smallest surface tested in the experimental series was 0.808 m², yielding a success rate of 97%.
- Below the tested range, significant limitations were observed within the vision layer, primarily driven by the loss of semantic target features within the simulation environment.
- Consequently, the lower bound for the landing surface area under the current architecture is established at 0.808 m².

Success rates:



There are 694 successful landings and 6 failures.

The overall success rate is 99.1 percent on average.

System Accuracy

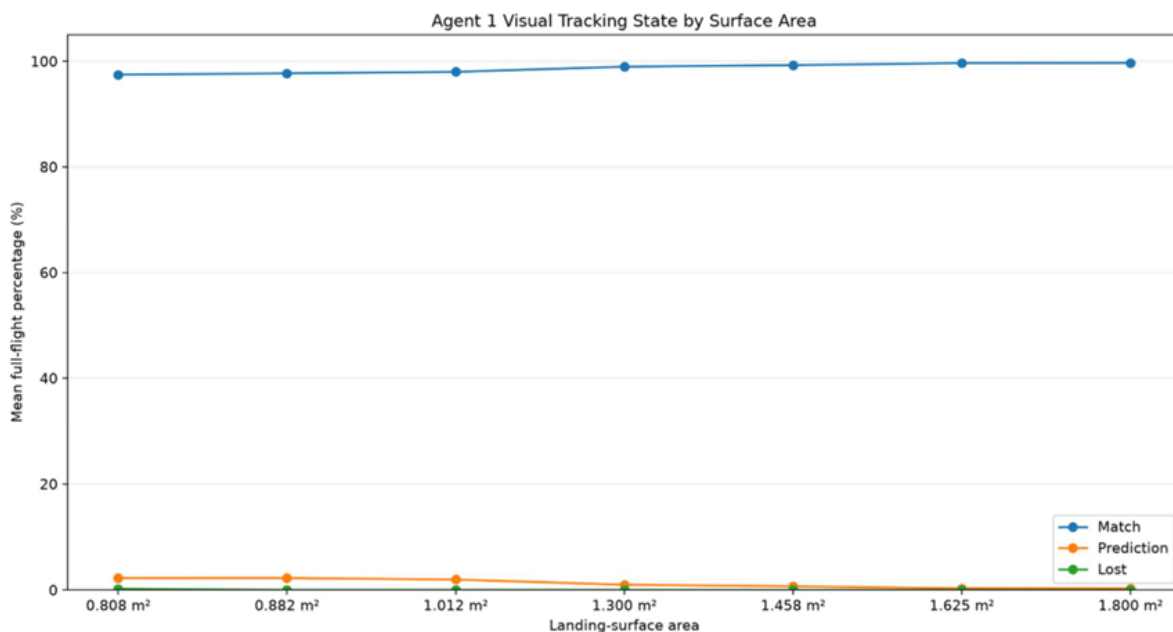
Since the system is divided into two agents, system accuracy is evaluated across several metrics:

- The percentage of time the first agent detected and locked onto the target from the moment of selection until handover to the second agent, indicating that the drone properly aligned over the target and was ready for landing.
- The percentage of time the second agent detected and locked onto the target from the moment of receiving responsibility from the first agent until the target's identity was lost due to extreme proximity, where alternative sensing- and control-based mechanisms compensated for the lack of visual feedback.
- The Euclidean radius/distance error of the deviation from the target center at the moment of contact between the drone and the target, measuring the distance in centimeters from the center.
- The average time, in seconds, required for the drone from target selection until actual touch-down.

- The average time required for the drone to be ready to initiate contact from the moment of target selection.

Vision Metrics

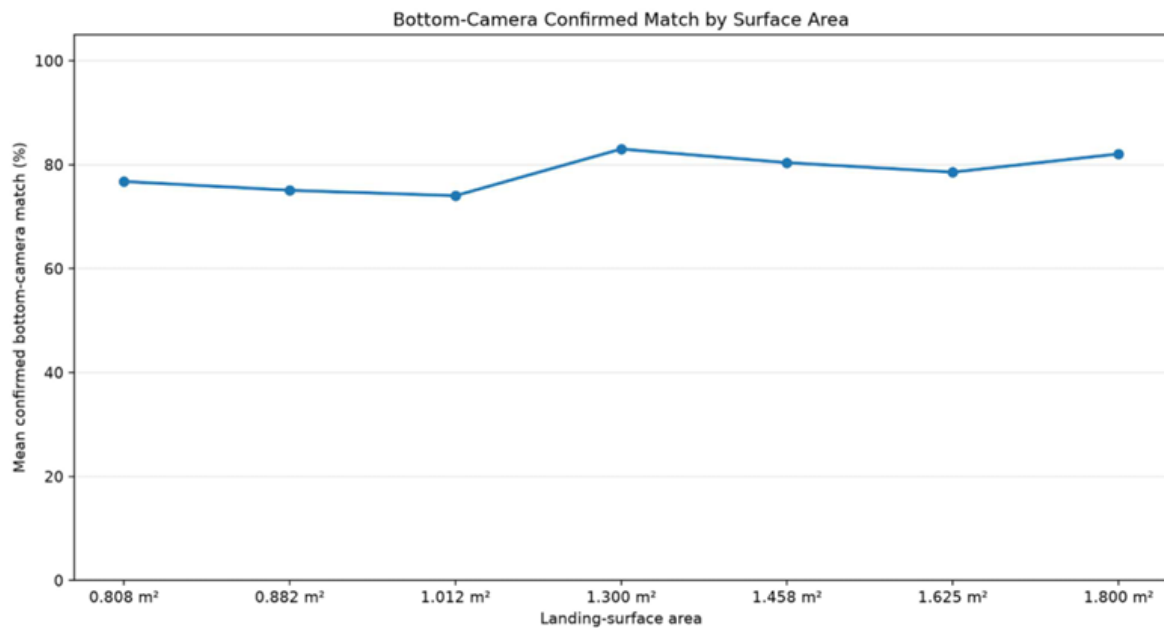
- The percentage of time the first agent detected and locked onto the target from the moment of selection until the handover of responsibility to the second agent meaning the drone aligned properly above the target and was ready to land.



It can be observed that the system's vision-based tracking mechanism maintains verified matching with the target for approximately 97.5% to 99.7% of the flight duration, depending on the target size.

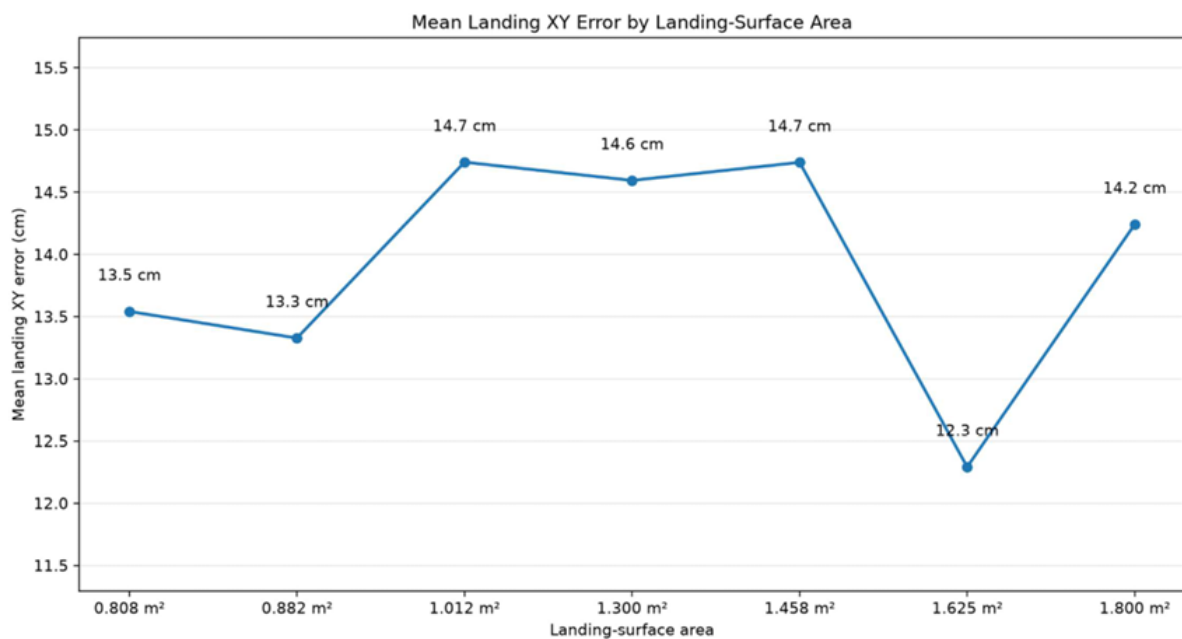
A general trend of improvement in the matching percentage can be identified as the target surface area increases.

The percentage of time the second agent detected and locked onto the target from the moment of receiving responsibility from the first agent until the moment it was so close to the target that its identity was lost, at which point alternative sensing and control based mechanisms compensated for the lack of visual feedback.



It can be observed that the tracking mechanism during the landing phase maintains an average of 80% of the landing duration. The reason this value is lower than the verified visibility time prior to the start of landing is that at a certain stage, due to close proximity to the target, many of the semantic features identified by the tracking mechanism during the flight have already disappeared, and at this stage there is no high verification of the target

Centering Index / Deviation Radius:



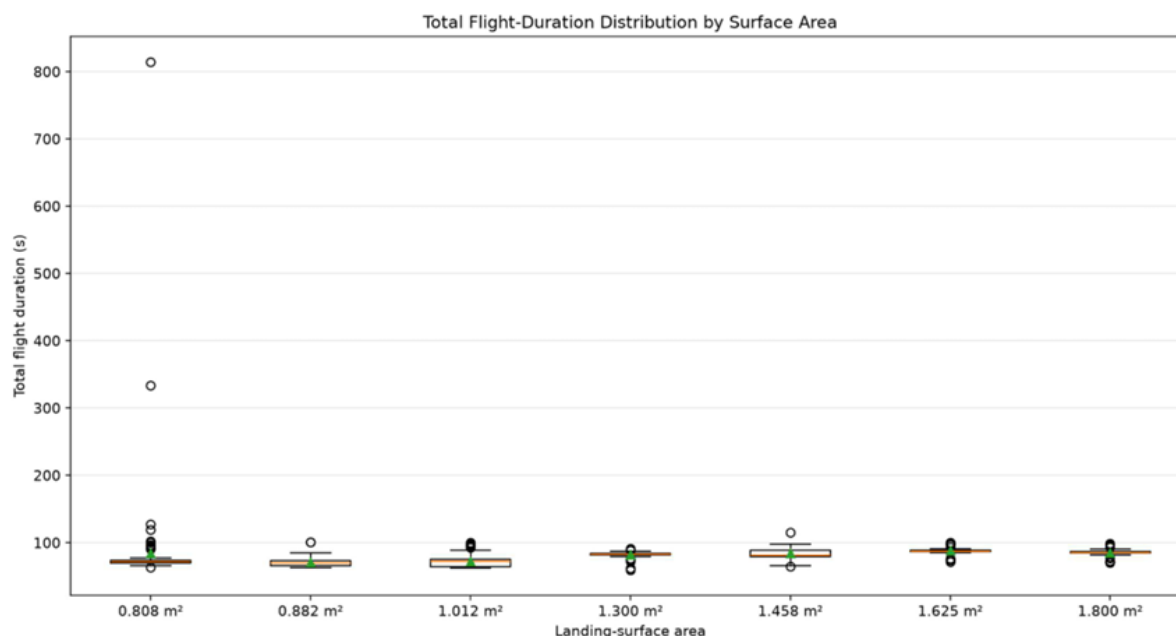
It can be observed from the data that, consistently, the average deviation radius in successful landings ranges between 12.3 and 14.7 cm.

This is an important metric because it provides a good indication of system accuracy, which is invariant, unrelated, or independent of the surface size.

For example, it can be observed that the deviation radius from the center of a 0.808 m^2 surface was smaller compared to a larger 1.800 m^2 surface.

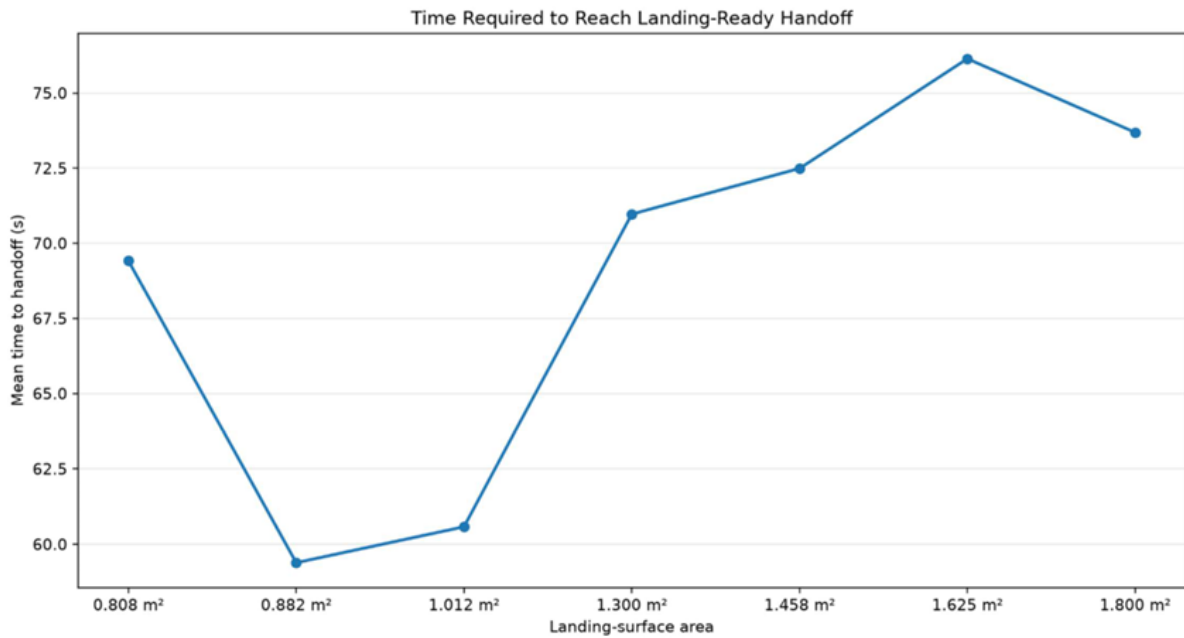
Time indicators:

Time from targeting to landing



It can be seen from the data that the average flight time from selection to contact with the target is about 80 seconds. There are exceptions resulting from failed landing attempts.

Average time to produce contact:



That is, the majority of the flight time was primarily dedicated to pursuing the target and preparing for landing (aligning and centering over it), while the remaining time was allocated to descent, corrections, and establishing contact.

System Weaknesses and Limitations

One of the core limitations observed in the system lies within the Vision layer, particularly when the target occupies a small footprint within the image.

This limitation may partly stem from the simulation environment providing less rich and diverse visual data compared to the real world, where deeper, richer visual details, textures, and features exist to assist in detection and tracking.

Nevertheless, this remains a hypothesis based on experiments and comparisons conducted between testing the tracking system on proactively collected real-world data and targets, versus the simulation-based experiments.

The time required to reach landing is non-constant, varying according to the initial position, target trajectory, and environmental conditions.

What Would We Improve?

Should the project be continued, future efforts would likely involve conducting more complex observations such as winding paths featuring various obstacles, tunnels, and bridges while correspondingly enhancing the tracking mechanism and improving the capacity to execute more sophisticated maneuvers and complex decision-making in such operational scenarios



GITHUB URL:

The updated files can be found under "Deployment" branch.

<https://github.com/talefraim-dev/Auto-drone-landing-on-a-moving-target>